Confluent Platform SQL Deployment via Flink SQL Client — Scalar UDFs

> This example deploys **two scalar UDFs** (source in examples/scalar_udf/java/) by submitting SQL statements through the **Flink SQL Client** running directly on the JobManager pod: `celsius_to_fahrenheit` and `fahrenheit_to_celsius`. Both UDFs ship in the same uber JAR and drive two independent long-running Flink jobs in the same session cluster, each reading from a Kafka source topic, applying the scalar conversion, and writing to a Kafka sink topic.

Table of Contents

- 1.0 Overview
  - 1.1 How this differs from the other deployment paths
- 2.0 How it works
  - 2.1 JAR delivery
  - 2.2 Kafka connector
  - 2.3 Statement flow
- 3.0 Prerequisites
- 4.0 How to run
  - 4.1 Deploy
  - 4.2 Monitor
  - 4.3 Tear down
- 5.0 Resources

1.0 Overview

1.1 How this differs from the other deployment paths

Aspect	**Confluent Platform SQL Client (this example)**	**Confluent Cloud Terraform**
Where it runs	Confluent Platform on Minikube	Confluent Cloud
How SQL is submitted	`sql-client.sh -f` on the JobManager pod	`confluent_flink_statement` Terraform resources
UDF JAR delivery	`kubectl exec` to Flink pods	`confluent_flink_artifact` (uploaded to CC)
Entry point	`make deploy-cp-scalar-udf`	`make deploy-cc-scalar-udf`
Requires code compilation	✅ (Java + Gradle for UDF JAR)	✅ (Java + Gradle for UDF JAR)
Statement lifecycle	Managed by Flink session cluster	Managed by Terraform state
Same codebase for both?	✅ (**same Java UDF code**, different Terraform vs SQL Client	✅ (**same Java UDF code**, different Terraform vs SQL Client for deployment)

Aspect	Confluent Platform SQL Client (this example)	Confluent Cloud Terraform
	for deployment)	

2.0 How it works

2.1 JAR delivery

On Confluent Platform there is no artifact store — the JAR must be physically present on the Flink pods.

The deploy script copies the uber JAR (built from examples/scalar_udf/java/) to `/opt/flink/usrlib/scalar-udf.jar` on every JobManager and TaskManager pod using `kubectl exec`. Both `CREATE FUNCTION ... USING JAR` statements (`celsius_to_fahrenheit` and `fahrenheit_to_celsius`) reference this same pod-local path, since both scalar UDFs are bundled in the same uber JAR.

2.2 Kafka connector

The CP Flink base image does not include the Kafka SQL connector. The FlinkDeployment uses two **init containers** to assemble `/opt/flink/lib/` at pod startup:

1. **copy-stock-lib** — copies the stock Flink lib JARs from the image into a shared `emptyDir` volume
2. **fetch-kafka-connector** — downloads `flink-sql-connector-kafka` from Maven Central into the same volume

The main Flink container then mounts this volume at `/opt/flink/lib/`, so the Kafka connector is on the classpath when the JVM starts.

2.3 Statement flow

The script pre-creates **all four Kafka topics**, then executes all SQL for **both** scalar UDFs in a single `sql-client.sh -f` session on the JobManager pod. Both UDFs are bundled in the same uber JAR, so a single pod-local JAR path feeds both `CREATE FUNCTION` statements.

```
┌──────────────────────────────────────────────────────────────────────────────
──┐
│  Pre-step: kafka-topics --create celsius_reading, celsius_to_fahrenheit,
│
│                                  fahrenheit_reading,
fahrenheit_to_celsius │
│
│
│  ── UDF 1: CelsiusToFahrenheit ──
│
│  Step  1:  DROP TABLE IF EXISTS celsius_reading                → OK
│
│  Step  2:  CREATE TABLE celsius_reading (... WITH kafka ...)   → OK
│
│  Step  3:  INSERT INTO celsius_reading VALUES (sample data)    →
```

```
submitted │
│  Step  4:  DROP TABLE IF EXISTS celsius_to_fahrenheit          → OK
│
│  Step  5:  CREATE TABLE celsius_to_fahrenheit (... WITH kafka) → OK
│
│  Step  6:  CREATE FUNCTION celsius_to_fahrenheit               → OK
│
│            AS 'scalar_udf.CelsiusToFahrenheit'
│
│            USING JAR 'file:///opt/flink/usrlib/scalar-udf.jar'
│
│  Step  7:  INSERT INTO celsius_to_fahrenheit                   →
submitted │
│            SELECT sensor_id, celsius_temperature,
│
│                   celsius_to_fahrenheit(celsius_temperature)
│
│              FROM celsius_reading
│
│
│
│  ── UDF 2: FahrenheitToCelsius ──
│
│  Step  8:  DROP TABLE IF EXISTS fahrenheit_reading             → OK
│
│  Step  9:  CREATE TABLE fahrenheit_reading (... WITH kafka)    → OK
│
│  Step 10:  INSERT INTO fahrenheit_reading VALUES (sample data) →
submitted │
│  Step 11:  DROP TABLE IF EXISTS fahrenheit_to_celsius          → OK
│
│  Step 12:  CREATE TABLE fahrenheit_to_celsius (... WITH kafka) → OK
│
│  Step 13:  CREATE FUNCTION fahrenheit_to_celsius               → OK
│
│            AS 'scalar_udf.FahrenheitToCelsius'
│
│            USING JAR 'file:///opt/flink/usrlib/scalar-udf.jar'
│
│            -- same JAR path as Step 6, two functions, one artifact
│
│  Step 14:  INSERT INTO fahrenheit_to_celsius                   →
submitted │
│            SELECT sensor_id, fahrenheit_temperature,
│
│                   fahrenheit_to_celsius(fahrenheit_temperature)
│
│              FROM fahrenheit_reading
│
└──────────────────────────────────────────────────────────────────────────────
──┘
```

Steps 7 and 14 are **two long-running streaming jobs** running side-by-side in the same Flink session cluster: Step 7 reads from `celsius_reading`, applies `celsius_to_fahrenheit`, and writes to `celsius_to_fahrenheit`; Step 14 reads from `fahrenheit_reading`, applies `fahrenheit_to_celsius`, and writes to `fahrenheit_to_celsius`.

3.0 Prerequisites

- macOS with Homebrew or Linux with apt-get, and Docker Desktop running
- Java 21 and Gradle installed (for building the UDF JAR)
- Confluent Platform and Flink stack already deployed via:

```
make install-prereqs       # installs tooling (docker, kubectl, minikube,
helm, gradle)
make cp-up                 # Minikube → CFK Operator → Kafka + SR + Connect
+ C3
make cp-watch              # watch pods come up (Ctrl+C when all Running)
make flink-up              # cert-manager → Flink Operator → CMF → Flink
session cluster
make flink-status          # verify Flink pods are Running
```

4.0 How to run

All commands are run from the **project root** (where the `Makefile` lives).

4.1 Deploy

A single target builds the UDF JAR, copies it to the Flink pods, and executes all SQL:

```
make deploy-cp-scalar-udf
```

Behind the scenes this runs:

Step	What it does
1	`./gradlew clean shadowJar` — builds the UDF uber JAR from `examples/scalar_udf/java/`
2	`kubectl exec` — copies the JAR to all JobManager and TaskManager pods
3	`kafka-topics --create` — pre-creates Kafka topics (`celsius_reading`, `celsius_to_fahrenheit`, `fahrenheit_reading`, `fahrenheit_to_celsius`)
4	`sql-client.sh -f` — executes all SQL statements for **both** UDFs in a single session on the JobManager pod

4.2 Monitor

Open the Flink Dashboard to see the **two running streaming jobs** (Celsius → Fahrenheit and Fahrenheit → Celsius):

```
make flink-ui              # opens http://localhost:8081
```

4.3 Tear down

To stop the running streaming jobs and drop all tables and functions:

```
make teardown-cp-scalar-udf
```

This cancels any running Flink jobs via the Flink REST API, submits `DROP FUNCTION` and `DROP TABLE` statements for **both** UDFs (`celsius_to_fahrenheit`, `fahrenheit_to_celsius`) and all four tables (`celsius_reading`, `celsius_to_fahrenheit`, `fahrenheit_reading`, `fahrenheit_to_celsius`), and deletes the four Kafka topics.

5.0 Resources

- Flink SQL Client
- Flink Kafka Connector
- User-defined Functions (Flink)
- Create a User-Defined Function (Confluent Cloud)